

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Resumen Teórico

Triggers y Procedimientos Almacenados
(Server Programming) — Práctico N.º 3

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Procedimientos almacenados: visión general	2
2.1. Ventajas y desventajas	2
3. Procedimientos almacenados en MySQL (SQL/PSM)	3
3.1. Estructura del lenguaje	3
3.2. Variables locales y asignación	3
3.3. <i>Handlers</i> y manejo de excepciones	3
3.4. Cursores	4
3.5. Sentencias de control de flujo	4
3.6. Ejemplo completo: cursor + handler + REPEAT	5
3.7. Modificación e invocación	5
4. Procedimientos y funciones en PostgreSQL (PL/pgSQL)	6
4.1. Estructura de PL/pgSQL	6
4.2. Parámetros y variables	6
4.3. Bloques anidados y control de flujo	6
4.4. Cursores explícitos	7
4.5. Excepciones	7
4.6. Invocación	8
5. Procedimientos y funciones en Oracle (PL/SQL)	8
5.1. Sintaxis general	8
5.2. Excepciones en Oracle	8
5.3. Cursores en PL/SQL	9
5.4. Invocación en Oracle	9
6. Triggers (disparadores)	10
6.1. Variables contextuales NEW y OLD	10
6.2. Estándar y dialectos por motor	10
6.3. Triggers en MySQL	10
6.4. Triggers en PostgreSQL	11
6.5. Triggers en Oracle	11
7. Reglas de reescritura en PostgreSQL	12
8. Triggers vs procedimientos	13
9. Conexión con el Práctico 3	13
10. Ideas clave para repasar antes del parcial	13
Resumen final	14

1. Introducción

Hasta el Práctico 2, todo lo que escribíamos en SQL se ejecutaba como una sentencia suelta enviada por el cliente. El Práctico 3 introduce la idea de **programación del lado servidor** (*server programming*): bloques de código que viven dentro del motor y se disparan al ser invocados por un cliente o, automáticamente, ante ciertos eventos sobre las tablas. Hay dos grandes herramientas:

Procedimientos y funciones almacenadas

Programas con SQL embebido, parámetros, variables, control de flujo y manejo de errores. Se invocan explícitamente desde el cliente con **CALL** (MySQL/PostgreSQL) o desde un bloque **BEGIN ... END** (Oracle).

Triggers (disparadores)

Procedimientos que el motor ejecuta *automáticamente* cuando ocurre un evento sobre una tabla (típicamente **INSERT/UPDATE/DELETE**). Son ideales para auditoría, mantener consistencia entre tablas relacionadas y normalización de datos antes de su persistencia.

Idea clave. El estándar SQL:1999 incorpora *stored procedures* y *triggers*, pero cada motor mantiene su propio lenguaje: **PL/SQL** en Oracle, **PL/pgSQL** en PostgreSQL, **SQL/PSM** (sintaxis SQL:2003) en MySQL. La lógica conceptual es la misma; lo que cambia es la sintaxis y los detalles de cada motor.

2. Procedimientos almacenados: visión general

Un **procedimiento almacenado** (*stored procedure*) es un programa que contiene comandos SQL y se almacena dentro del servidor de la base de datos. Posee parámetros de entrada y salida y puede ser invocado por un cliente como si fuera una rutina externa, pero corre *dentro* del motor.

Los procedimientos están disponibles desde el estándar SQL:1999. PostgreSQL incluso permite escribirlos en lenguajes externos (C, Java, Python, etc.). En este resumen nos centramos en los lenguajes “nativos” que son los que se usan en el práctico:

- **MySQL** sigue la sintaxis **SQL:2003** (lenguaje *SQL/PSM*).
- **PostgreSQL** usa **PL/pgSQL**.
- **Oracle** usa **PL/SQL**.

2.1. Ventajas y desventajas

Ventaja	Por qué importa
Rendimiento	Se ejecutan en el motor, sin viaje de datos al cliente.
Compilación previa	Quedan analizados y optimizados al crearse, no en cada llamada.
Menos tráfico de red	Una sola llamada en lugar de N sentencias.
Centralización	Forman parte de la base; siempre están disponibles.
Seguridad	Se puede otorgar EXECUTE sobre el procedimiento sin dar permisos sobre las tablas que toca.
Encapsulamiento	El cliente invoca “ <i>eliminar cliente</i> ” sin conocer los detalles internos.

Desventaja principal. Cada motor tiene su propio dialecto y, por lo tanto, migrar lógica entre motores es trabajoso. Es el precio del poder.

3. Procedimientos almacenados en MySQL (SQL/PSM)

MySQL incorpora *triggers* y *stored procedures* desde la versión 5. La sintaxis básica es:

```
CREATE PROCEDURE sp_name ([parameter [, ...]] [characteristic ...]
    routine_body;

CREATE FUNCTION sp_name ([parameter [, ...]] RETURNS type
    [characteristic ...] routine_body;

parameter:      [IN | OUT | INOUT] param_name type
characteristic: LANGUAGE SQL
                | { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
                | SQL SECURITY { DEFINER | INVOKER }
                | COMMENT 'string'
```

Parámetros IN/OUT/INOUT. IN es entrada (por valor), OUT es de salida (escrito por la rutina), e INOUT se comporta como ambos. Si no se especifica el modo, MySQL asume IN.

Características. CONTAINS SQL indica que el cuerpo no lee ni modifica datos; READS SQL DATA sólo lee; MODIFIES SQL DATA puede modificar. SQL SECURITY DEFINER (default) ejecuta con permisos de quien creó la rutina; INVOKER con los del invocador.

3.1. Estructura del lenguaje

El *routine_body* se construye con los siguientes elementos:

- Sentencias compuestas BEGIN ... END (bloques).
- Sentencias DECLARE para variables locales, condiciones, *handlers* y cursores.
- Asignación de variables (SET, SELECT ... INTO).
- Sentencias de control de flujo: IF, CASE, LOOP, LEAVE, ITERATE, REPEAT, WHILE.
- Cursores para recorrer resultados de SELECT fila por fila.
- Manejo de excepciones mediante *handlers*.

Bloques BEGIN ... END. Todo procedimiento no trivial está delimitado por BEGIN ... END. Pueden anidarse: cada subbloque tiene su propio scope de variables, sus declaraciones y sus *handlers*.

3.2. Variables locales y asignación

```
-- Declaracion (en el orden correcto: declaraciones siempre arriba del bloque).
DECLARE var_name type [DEFAULT value];
DECLARE x INT;
DECLARE saldo DECIMAL(12,2) DEFAULT 0;

-- Asignacion clasica.
SET x = 10;

-- Asignacion via SELECT.
SELECT MAX(precio) INTO x FROM producto;
```

3.3. Handlers y manejo de excepciones

Los *handlers* reaccionan ante condiciones (errores o avisos) durante la ejecución:

```
DECLARE handler_type HANDLER FOR condition_value [, ...] sp_statement;

handler_type:    CONTINUE | EXIT | UNDO
condition_value: SQLSTATE sqlstate_value
                  | condition_name
                  | SQLWARNING
                  | NOT FOUND
                  | SQLEXCEPTION
                  | mysql_error_code
```

Tipos de *handler*.

- CONTINUE: ejecuta la sentencia del handler y continúa con la siguiente sentencia del programa.
- EXIT: ejecuta el handler y *termina* el bloque que lo contiene.
- UNDO: deshace lo hecho dentro del bloque (no siempre soportado).

Tipos de condición.

- SQLSTATE 'xxxxx': estándar, comparten todos los motores.
- SQLWARNING: cualquier SQLSTATE que empiece con 01.
- NOT FOUND: cualquier SQLSTATE que empiece con 02. Es el típico “ya no hay más filas” de un cursor.
- SQLEXCEPTION: todos los SQLSTATE restantes.
- mysql_error_code: códigos específicos de MySQL.

Lanzar excepciones del usuario. Desde MySQL 5.5:

```
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'lanzo una excepcion';
```

3.4. Cursores

Un *cursor* permite recorrer fila por fila el resultado de una consulta. Su uso típico tiene cuatro etapas: DECLARE, OPEN, repetidos FETCH, CLOSE.

```
-- Declaracion.
DECLARE cursor_name CURSOR FOR select_statement;

-- Operaciones.
OPEN cursor_name;
FETCH cursor_name INTO var1 [, var2, ...];
CLOSE cursor_name;
```

3.5. Sentencias de control de flujo

- IF cond THEN ... ELSE ... END IF;
- CASE ... WHEN ... THEN ... END CASE;
- LOOP ... END LOOP; (con LEAVE para salir e ITERATE para volver al principio).
- REPEAT ... UNTIL cond END REPEAT;
- WHILE cond DO ... END WHILE;

REPEAT (semántica). Las sentencias del bloque se ejecutan **al menos una vez**, y se repiten hasta que la condición `search_condition` se vuelva verdadera.

```
REPEAT
    statement_list
UNTIL search_condition
END REPEAT;
```

3.6. Ejemplo completo: cursor + handler + REPEAT

Recorre `tabla1` filtrando por un parámetro y, según una comparación entre dos columnas, inserta en `tabla2` o en `tabla3`:

```
CREATE PROCEDURE cursordemo(IN parametro INT)
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE a1 CHAR(16);
    DECLARE b1, c1 INT;

    DECLARE cursor1 CURSOR FOR
        SELECT a, b, c FROM tabla1 WHERE id < parametro;

    -- Cuando NOT FOUND (SQLSTATE 02000), termina el recorrido.
    DECLARE CONTINUE HANDLER FOR SQLSTATE '02000'
    BEGIN
        SET done = 1;
    END;

    OPEN cursor1;
    REPEAT
        FETCH cursor1 INTO a1, b1, c1;
        IF NOT done THEN
            IF b1 < c1 THEN
                INSERT INTO tabla2 VALUES (a1, b1);
            ELSE
                INSERT INTO tabla3 VALUES (a1, c1);
            END IF;
        END IF;
    UNTIL done END REPEAT;
    CLOSE cursor1;
END;
```

Lectura del ejemplo. El *handler* de tipo `CONTINUE` captura el aviso “no hay más filas” y pone `done = 1`. El bucle `REPEAT` hace `FETCH` dentro y termina cuando `done` pasa a 1.

3.7. Modificación e invocación

Para cambiar las características de una rutina existente (por ejemplo, cambiar de `DEFINER` a `INVOKER`, o agregar `COMMENT`):

```
ALTER {PROCEDURE | FUNCTION} sp_name [characteristic ...];
```

Para invocar un procedimiento se usa `CALL`:

```
CALL nombreProcedimiento([parametroActual [, ...]]);
CALL cursordemo(2);
```

4. Procedimientos y funciones en PostgreSQL (PL/pgSQL)

PostgreSQL implementa funciones (CREATE FUNCTION) y, desde la versión 11, también *procedures* (CREATE PROCEDURE). Pueden escribirse en distintos lenguajes; el de uso más común es PL/pgSQL, orientado a bloques.

4.1. Estructura de PL/pgSQL

```
CREATE [OR REPLACE] FUNCTION nombreFuncion ([parametro [, ...]])
RETURNS tipoRetorno AS $$
[DECLARE
    declaracion_de_variables_locales]
BEGIN
    sentencias_propias_de_la_funcion;
[EXCEPTION
    WHEN condicion [OR condicion ...]
        THEN sentencias_para_manejar_la_excepcion
    [WHEN condicion [OR condicion ...]
        THEN sentencias_para_manejar_la_excepcion ...]
]
END;
$$ LANGUAGE 'plpgsql';
```

Delimitadores \$\$. El cuerpo entero se encierra entre delimitadores “dólar comillado”. PostgreSQL acepta \$nombre\$... \$nombre\$ si se necesita anidar.

4.2. Parámetros y variables

```
-- Parametros: [IN | OUT | INOUT] [nombre] tipo
CREATE FUNCTION sum_n_product(int, y int, OUT sum int, OUT prod int) AS $$
BEGIN
    sum := $1 + y;      -- $1 referencia al primer parametro sin nombre.
    prod := $1 * y;
END;
$$ LANGUAGE plpgsql;
```

Si una función se declara con uno o más OUT, PostgreSQL la trata como devolviendo una fila con esas columnas. Se invoca con un SELECT.

Declaración de variables.

```
nombre [CONSTANT] tipo [NOT NULL] [{ DEFAULT | := } expresion];

-- Alias de un parametro posicional.
nombreAlias ALIAS FOR $n;

-- Asignacion.
nombreVariable := expresion;
```

4.3. Bloques anidados y control de flujo

Igual que en MySQL, los bloques se delimitan con BEGIN ... END y pueden anidarse, cada uno con sus propios manejadores de excepciones.

PL/pgSQL provee:

- LOOP ... END LOOP;
- CASE

- EXIT (sale del bloque/ciclo)
- IF ... THEN ... ELSE ... END IF;
- FOR
- WHILE

Sentencia FOR numérica.

```
FOR variable IN inicio..fin LOOP
    cuerpo_del_for;
END LOOP;

-- Iteracion en sentido inverso.
FOR variable IN REVERSE fin..inicio LOOP
    cuerpo_del_for;
END LOOP;

-- Ejemplo: factorial.
FOR inc IN 1..10 LOOP
    factorial := factorial * inc;
END LOOP;
```

Sentencia FOR sobre filas (cursor implícito). PL/pgSQL permite recorrer un SELECT sin abrir el cursor manualmente:

```
DECLARE
    registro RECORD;          -- registro generico
BEGIN
    FOR registro IN SELECT * FROM productos LOOP
        stock := registro.stock_actual + 100;
    END LOOP;
END;
```

Tipos de registro. Se puede declarar un registro RECORD (genérico) o ligado al esquema de una tabla con `persona%ROWTYPE`. Esto último brinda chequeo de tipos por columna.

4.4. Cursores explícitos

```
DECLARE nombre CURSOR FOR sentenciaSelect;

-- Operaciones: FETCH, CLOSE, MOVE.
-- La variable booleana FOUND indica si la ultima operacion devolvio fila.
```

4.5. Excepciones

PostgreSQL permite lanzar excepciones del usuario:

```
RAISE EXCEPTION 'el mensaje %', valor;
```

Y capturarlas con bloques EXCEPTION ... WHEN:

```
BEGIN
    -- ...
EXCEPTION
    WHEN condicion [OR condicion ...] THEN
        sentencias_para_manejar_la_excepcion;
END;
```

4.6. Invocación

A diferencia de MySQL, en PostgreSQL las funciones se invocan con **SELECT**:

```
SELECT nombreProcedimiento([parametroActual [, ...]]);
SELECT procedimiento(2);
```

Si la función tiene parámetros OUT, conviene **SELECT * FROM funcion(...)** para ver todas las columnas devueltas.

5. Procedimientos y funciones en Oracle (PL/SQL)

Oracle ofrece un lenguaje muy potente, **PL/SQL**, con estructura semejante a la de PL/pgSQL.

5.1. Sintaxis general

Procedimiento.

```
CREATE [OR REPLACE] PROCEDURE nombreProc ([parametro [, ...]])
AS
    [DECLARE]
        declaraciones
BEGIN
    programa;
[EXCEPTION]
    tratamiento_de_excepciones]
END;
/
```

Función.

```
CREATE [OR REPLACE] FUNCTION nombreFunc ([parametro [, ...]])
RETURN tipo
AS
    [declaraciones]
BEGIN
    programa;
[EXCEPTION]
    tratamiento_de_excepciones]
END;
/
```

Parámetros.

```
nombreParametro [IN | OUT | IN OUT] tipo [DEFAULT expr]
```

Variables.

```
nombreVariable tipo [NOT NULL] [:= valor | DEFAULT valor];
nombreVariable := expresion;          -- asignacion
```

5.2. Excepciones en Oracle

Oracle permite lanzar excepciones del usuario con **RAISE_APPLICATION_ERROR**, cuyo número debe estar en el rango $[-20999, -20000]$:

```
RAISE_APPLICATION_ERROR(Error_number INT, texto TEXT);

-- Ejemplo.
RAISE_APPLICATION_ERROR(-20011, 'prueba de excepcion');
```

También se puede declarar una excepción con nombre y lanzarla con **RAISE**.

Captura de excepciones. Con `WHEN OTHERS THEN ...`, y las funciones `SQLCODE` y `SQLERRM` para obtener el código y el mensaje:

```
DECLARE
    err_num NUMBER;
    err_msg VARCHAR2(255);
    result NUMBER;
BEGIN
    RAISE_APPLICATION_ERROR(-20011, 'prueba de excepcion');
EXCEPTION
    WHEN OTHERS THEN
        err_num := SQLCODE;
        err_msg := SQLERRM;
        DBMS_OUTPUT.PUT_LINE('Error: ' || TO_CHAR(err_num));
        DBMS_OUTPUT.PUT_LINE(err_msg);
END;
```

5.3. Cursores en PL/SQL

La sintaxis de `DECLARE / OPEN / FETCH / CLOSE` es la misma que en MySQL. La diferencia interesante son los *atributos* que expone cada cursor:

- `%FOUND`: true si el último `FETCH` devolvió fila.
- `%NOTFOUND`: complemento del anterior.
- `%ISOPEN`: true si el cursor está abierto.
- `%ROWCOUNT`: cantidad de filas leídas hasta el momento.

Ejemplo de manejo de cursor.

```
CREATE OR REPLACE PROCEDURE listar IS
    CURSOR cproductos IS SELECT nProd, precio FROM productos;
    regProducto cproductos%ROWTYPE;
BEGIN
    OPEN cproductos;
    FETCH cproductos INTO regProducto;
    WHILE cproductos%FOUND LOOP
        DBMS_OUTPUT.PUT_LINE(regProducto.precio);
        FETCH cproductos INTO regProducto;
    END LOOP;
    CLOSE cproductos;
END;
/
```

5.4. Invocación en Oracle

Se invoca dentro de un bloque anónimo:

```
[DECLARE
    declaraciones]
BEGIN
    nombreProc([parametroActual [, ...]]);
END;
/

-- Ejemplo.
DECLARE
    nombre_parametro INT;
BEGIN
    nombre_parametro := 3;
    pruebaProc(3, nombre_parametro);
```

```
END;
/
```

6. Triggers (disparadores)

Un **trigger** es un procedimiento que el motor ejecuta *automáticamente* cuando se cumple una *condición de disparo*: una operación DML (INSERT, UPDATE o DELETE) sobre una tabla específica. Se utilizan principalmente para:

- generar información de auditoría;
- mantener consistencia entre tablas;
- normalizar datos antes de su persistencia (mayúsculas, TRIM, etc.);
- completar campos derivados (autonumerado en motores que no lo traen nativo).

Para diseñar un trigger hay que definir:

1. la **condición de disparo** (qué evento, sobre qué tabla, antes o después);
2. las **acciones** a ejecutar cuando se dispara.

6.1. Variables contextuales NEW y OLD

Dentro del cuerpo del trigger, dos variables especiales dan acceso a la fila afectada:

- NEW: contiene los *nuevos* valores. Disponible en INSERT y UPDATE.
- OLD: contiene los *viejos* valores. Disponible en UPDATE y DELETE.

En PostgreSQL y MySQL se referencian directamente como NEW.col/OLD.col; en Oracle el cuerpo PL/SQL las referencia con dos puntos: :NEW.col/:OLD.col.

6.2. Estándar y dialectos por motor

- SQL-92 *no* incluye triggers; cada motor tenía su propia forma.
- SQL:1999 incorporó la sintaxis estándar, pero los motores siguen ofreciendo extensiones.
- El *lenguaje* del cuerpo del trigger es el mismo que para procedimientos almacenados (SQL/PSM, PL/pgSQL o PL/SQL).

6.3. Triggers en MySQL

Disponibles desde MySQL 5.0.2.

```
CREATE TRIGGER nombre_trigger momento_disparo evento_disparo
  ON nombre_tabla
  FOR EACH ROW
  sentencia_trigger;

-- momento_disparo: BEFORE / AFTER
-- evento_disparo:  INSERT / UPDATE / DELETE
```

Ejemplo: auditar bajas en artículos.

```
delimiter $$
CREATE TRIGGER trigger_baja_articulos
AFTER DELETE ON articulos
FOR EACH ROW
BEGIN
  INSERT INTO articulos_bajas
```

```
VALUES (OLD.nart, OLD.descr, OLD.precio, CURRENT_USER(), NOW());
END;
$$
delimiter ;
```

CURRENT_USER() vs usuario de aplicación. En MySQL, `CURRENT_USER()` devuelve el usuario con el que se estableció la conexión (típicamente uno técnico de la aplicación, como `webapp@%`). Casi siempre la auditoría busca otra cosa: el usuario *de la aplicación cliente* (la persona logueada). Eso debe registrarse en una variable de sesión:

```
-- Setear desde el cliente al iniciar la sesion.
SET @id_usuario = 4;

-- El trigger lee la variable de sesion.
INSERT INTO AUDITORIA VALUES (0, @id_usuario, NEW.fecha);
```

6.4. Triggers en PostgreSQL

PostgreSQL desacopla *trigger* y *función*: primero se crea una función que devuelve `TRIGGER`, después se asocia esa función a una tabla con `CREATE TRIGGER`.

```
CREATE TRIGGER nombre {BEFORE | AFTER}
    {evento [OR ...]}
    ON tabla [FOR [EACH] {ROW | STATEMENT}]
    EXECUTE PROCEDURE nombreFunc(argumentos);
```

Ejemplo de auditoría.

```
-- 1) Funcion que se ejecutara al disparar el trigger.
CREATE FUNCTION funcion_auditoria() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO auditoria
    VALUES (1, OLD.nart, NOW(), OLD.cant - NEW.cant);
    RETURN NEW;
END;
$$ LANGUAGE 'plpgsql';

-- 2) Asociar el trigger a la tabla.
CREATE TRIGGER trigger_auditoria
AFTER UPDATE ON articulos
FOR EACH ROW
EXECUTE PROCEDURE funcion_auditoria();
```

Diferencia ROW vs STATEMENT. Con `FOR EACH ROW` la función se ejecuta una vez por cada fila afectada; con `FOR EACH STATEMENT`, una sola vez por sentencia.

6.5. Triggers en Oracle

```
CREATE [OR REPLACE] TRIGGER [esquema.]nombre
{BEFORE | AFTER | INSTEAD OF}
{ dml_event_clause
| ddl_event [OR ddl_eventN]...
| database_event [OR database_eventN]...}
ON {tabla | view | [esquema.]SCHEMA | DATABASE}
[FOR EACH ROW]
[WHEN (condicion)]
{pl/sql_block | call_procedure_statement};
```

Ejemplo (equivalente al de MySQL).

```
CREATE TRIGGER trigger_baja_articulos
AFTER DELETE ON articulos
FOR EACH ROW
BEGIN
    INSERT INTO articulos_bajas
    VALUES (:OLD.nart, :OLD.descr, :OLD.precio, USER, SYSDATE);
END;
/
```

INSTEAD OF. Es propio de Oracle/PostgreSQL y se asocia a *vistas*: en lugar de ejecutar la operación sobre la vista (que muchas veces no es actualizable), el trigger redirige la acción a las tablas base.

WHEN (condicion). Permite filtrar el disparo: sólo se ejecuta el cuerpo si la condición sobre NEW/OLD es verdadera. Es lo que permite, por ejemplo, autonumerar sólo cuando el cliente no provee un valor explícito:

```
CREATE OR REPLACE TRIGGER bi_parquimetro_autonumerado
BEFORE INSERT ON parquemetro
FOR EACH ROW
WHEN (NEW.id_parquimetro IS NULL)
BEGIN
    SELECT seq_parquimetro.NEXTVAL
    INTO :NEW.id_parquimetro
    FROM dual;
END;
/
```

7. Reglas de reescritura en PostgreSQL

PostgreSQL incluye un mecanismo extra, distinto de los triggers, llamado **sistema de reglas de reescritura** (CREATE RULE). En lugar de ejecutar código adicional cuando ocurre un evento, *modifica el query* antes de ejecutarlo.

Las vistas son reglas. En PostgreSQL, CREATE VIEW vista AS SELECT * FROM tabla; es equivalente a:

```
CREATE TABLE vista (lista_de_columnas);
CREATE RULE ReescribeVista AS ON SELECT TO vista
DO INSTEAD SELECT * FROM tabla;
```

es decir, una tabla vacía más una regla que reescribe los SELECT a esa tabla.

Reglas de actualización. Permiten interceptar INSERT, UPDATE o DELETE:

```
CREATE [OR REPLACE] RULE nombre
AS ON evento TO tabla
[WHERE condicion]
DO [ALSO | INSTEAD]
{NOTHING | comando | (comando; comando ...)};
```

Ejemplo. Auditoría de cambios de precio:

```
CREATE RULE log_precio_productos
AS ON UPDATE TO productos
WHERE NEW.precio <> OLD.precio
DO ALSO
INSERT INTO productos_log
VALUES (NEW.codigo, NEW.nombre, OLD.precio, NEW.precio,
       current_user, current_timestamp);
```

8. Triggers vs procedimientos

	Procedimiento almacenado	Trigger
Quién lo dispara	El cliente, explícitamente, con <code>CALL/SELECT</code>	El motor, <i>automáticamente</i> , ante un evento DML
Parámetros	Sí (IN, OUT, INOUT)	No: el “input” es la fila afectada (NEW/OLD)
Frecuencia	Una vez por invocación	Por fila o por sentencia, según la configuración
Uso típico	Lógica de negocio invocable, encapsulamiento	Auditoría, integridad cruzada, normalización, autonumerado
Aborta operación	Sólo levantando excepción	Sí, levantando excepción en BEFORE

9. Conexión con el Práctico 3

- **Ejercicio 1 (MySQL/PostgreSQL):** agrega `cantidad_maxima_items` a factura y la valida con triggers BEFORE INSERT/UPDATE, levantando excepción con `SIGNAL SQLSTATE '45000'` (MySQL) o `RAISE EXCEPTION` (PostgreSQL) si se supera el máximo.
- **Ejercicio 2 (PostgreSQL):** función con parámetros IN/OUT que recorre las cotizaciones con un FOR ... LOOP (sin usar MAX). Aplica directamente la *sentencia FOR sobre filas* del teórico.
- **Ejercicio 3 (Oracle):** dos procedimientos almacenados con IN/OUT, validación con `RAISE_APPLICATION_ERROR`, manejo de `NO_DATA_FOUND` y salida con `DBMS_OUTPUT.PUT_LINE`.
- **Ejercicio 4 (MySQL):** procedimiento creado por `vendedor2` con `SQL SECURITY INVOKER`; conecta DCL del Práctico 2 con SQL/PSM del Práctico 3.
- **Ejercicio 5:** triggers de tres tipos: *normalización* (BEFORE INSERT, UPPER), *auditoría* (AFTER UPDATE, leyendo el usuario de aplicación de una variable de sesión) y *autonumerado* (AUTO_INCREMENT en MySQL, SEQUENCE + TRIGGER BEFORE INSERT WHEN (NEW.id IS NULL) en Oracle).

10. Ideas clave para repasar antes del parcial

1. **Procedimiento vs trigger.** El primero se invoca; el segundo se dispara automáticamente.
2. **Cada motor, su lenguaje:** SQL/PSM (MySQL), PL/pgSQL (PostgreSQL), PL/SQL (Oracle).
3. **Bloque BEGIN ... END** es la unidad básica de ejecución y de scope; puede anidarse.
4. **DECLARE primero, código después:** las variables, condiciones, *handlers* y cursores se declaran al inicio de cada bloque.

5. **Parámetros:** IN (entrada), OUT (salida), INOUT (ambos). En PostgreSQL, una función con OUT se invoca con `SELECT * FROM funcion(...)`.
6. **Excepciones** (cómo lanzarlas): `SIGNAL SQLSTATE '45000'` (MySQL), `RAISE EXCEPTION '...'` (PostgreSQL), `RAISE_APPLICATION_ERROR(-20..., '...')` (Oracle).
7. **Excepciones** (cómo capturarlas): *handlers* CONTINUE/EXIT/UNDO en MySQL, bloques `EXCEPTION WHEN ... THEN` en PostgreSQL/Oracle.
8. **Cursores:** ciclo `DECLARE` → `OPEN` → `FETCH` (en bucle) → `CLOSE`. La condición `NOT FOUND` (`SQLSTATE '02000'`) o el atributo `%NOTFOUND` indican fin de iteración.
9. **Cursor implícito en PL/pgSQL:** el bucle `FOR registro IN SELECT ... LOOP ... END LOOP` ahorra el `OPEN/FETCH/CLOSE` explícitos.
10. **Triggers** se diseñan con dos elementos: condición de disparo (`BEFORE/AFTER` + `INSERT/UPDATE/DELETE`) y cuerpo. Dentro: `NEW/OLD` (con `:` en Oracle).
11. **Auditar el usuario de aplicación** requiere variables de sesión, no `CURRENT_USER()`.
12. **PostgreSQL separa función y trigger:** la función `RETURNS TRIGGER`; el trigger se asocia con `CREATE TRIGGER ... EXECUTE PROCEDURE func()`.
13. **INSTEAD OF** es propio de vistas (Oracle y PostgreSQL): redirige una operación que se ejecuta sobre una vista hacia las tablas base.
14. **Reglas de reescritura** son una herramienta extra de PostgreSQL: *modifican el query*, no agregan código de ejecución.
15. **ALTER PROCEDURE/FUNCTION** cambia *características* (`SQL SECURITY`, `COMMENT`, etc.); para cambiar el cuerpo se usa `CREATE OR REPLACE`.

Resumen final

Los procedimientos almacenados y los triggers son las herramientas con las que la lógica de negocio se acomoda dentro del motor de base de datos. Compartiendo el mismo lenguaje del lado servidor en cada motor — SQL/PSM, PL/pgSQL o PL/SQL — permiten escribir código con parámetros, variables, control de flujo, cursores y excepciones, ejecutado lo más cerca posible de los datos.

Si quedan firmes los cinco bloques — *procedimiento vs función, parámetros IN/OUT, control de flujo y cursores, manejo de excepciones y triggers con NEW/OLD* —, el Práctico 3 se traduce directamente: cada inciso es una combinación de uno o dos de esos bloques sobre las bases de datos heredadas de los prácticos anteriores. La gracia no está sólo en escribir el código, sino en elegir bien cuándo usar trigger, cuándo procedimiento y cuándo ninguna de las dos (porque alcanza con un `CHECK` o una `FOREIGN KEY`).